



전자전기컴퓨터설계실험II

실험 후 보고서

순차논리회로 실습

실험 11-18

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 09월 26일
대상	업 / 다운 카운터 · 클록 분주 · 레지스터 · 시프트 레지스터 · PISO · Moore 상태 머신 · Mealy 상태 머신 · 7세그먼트 자리 스캔
검증환경	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView / Vivado 2026.1

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: <https://github.com/a40093632-ai/ece2-lab2>

제출 태그: [lab2-post-v1](#)

커밋 해시: [1c44fc7092a9451e124847c3c8cf73126d6ca7c4](#)

1. 실험 목적

본 실험에서는 8가지 순차논리회로를 사용한 기본 회로들의 동작을 익히는데 그 목적이 있다. Verilog HDL을 통해 회로를 구현하고 각 회로에 대해 VS Code 시뮬레이션과 Vivado Xsim 결과를 비교한다. 또한, 실제 FPGA 보드에 프로그래밍하여 스위치와 버튼 입력에 따른 LED 출력을 확인하여 회로가 정확하게 동작하고 있는지를 살핀다. 이를 통해 순차논리회로의 특성을 이해하고, 나아가 Verilog, Vivado, FPGA 보드의 사용법을 숙달하고자 한다.

2. 관련 이론

레지스터와 시프트 레지스터

레지스터는 여러 개의 플립플롭으로 구성되어 클록의 유효 에지에서 이진 데이터를 저장하는 순차논리회로이다. enable이 비활성화된 동안에는 기존 값을 유지하며, load 신호를 이용하면 병렬 입력 데이터를 한 번에 저장할 수 있다.^[1]

시프트 레지스터는 저장된 비트를 클록마다 한 방향으로 이동시키는 레지스터이다. 데이터의 입출력 방식에 따라 SISO(Serial-In Serial-Out), SIPO(Serial-In Parallel-Out), PISO(Parallel-In Serial-Out), PIPO(Parallel-In Parallel-Out)로 구분할 수 있다. SISO는 직렬 데이터를 그대로 지연시키는 데 사용되고, SIPO는 직렬 데이터를 병렬 형태로 변환한다. 반대로 PISO는 병렬 데이터를 저장한 뒤 한 비트씩 직렬로 내보내며, PIPO는 병렬 데이터의 저장과 전달에 사용된다.^[1] 본 실험에서는 PISO 회로가 포함 되어있다.

클록 분주와 시간 다중화

클록 분주기는 입력 클록의 주기를 세어 더 낮은 주파수의 신호를 생성하는 회로이다. 카운터가 정해진 횟수에 도달할 때 출력 상태를 바꾸거나 짧은 펄스를 만들 수 있으며, 이를 이용해 LED 점멸 속도나 다른 순차회로의 동작 시점을 제어할 수 있다.^[1]

여러 자리 7세그먼트 표시기는 한 번에 한 자리씩 매우 빠르게 선택하는 시간 다중화 방식으로 구동할 수 있다. 각 자리의 선택 신호와 세그먼트 데이터를 순차적으로 바꾸면, 사람의 눈에는 모든 자리가 동시에 켜진 것처럼 보인다. 본 실험에서는 8개 자리를 순차 스캔하고, 자리 전환 사이에 소등 구간을 두어 표시 잔상을 줄였다.

3. 실험 환경 및 공통 검증 방법

실험 환경

구분	내용
운영체제	Windows 11 64-bit

FPGA 설계 도구	AMD Vivado 2026.1
설계언어	Verilog HDL / SystemVerilog
코드 및 사전 검증	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView
FPGA 소자	xc7s75fgga484-1
회로 종류	순차논리회로
보드	Combo II-DLD S75 / 1kHz 기본 클록
결과 관리	Git 및 GitHub

공통 실행 순서

1. 동일한 RTL과 테스트 벤치를 Design Source와 Simulation Source에 각각 등록.
2. Behavioral Simulation을 실행하여 lab2_pass 검사 수와 입 출력 파형을 확인하고 VS Code 결과와 비교.
3. Synthesis와 Implementation을 실행한 뒤 Design Runs에서 완료 상태를 확인.
4. Utilization, DRC, Timing Summary를 확인하고 Generate Bitstream을 수행.
5. 생성된 비트 파일을 확인하고 FPGA Program Device를 한 뒤 스위치, 버튼, LED 및 7세그먼트 동작을 기록.
6. Vivado 화면, 보드 사진과 영상, bit 파일을 Github에 기록하고 보고서 작성.

실험별 설계파일

실험	Design Top	Simulation Top
11	lab2_counter	tb_counter4
12	lab2_clock_divider	tb_clock_divider
13	lab2_register	tb_register_pair
14	lab2_shift_register	tb_shift_register4
15	lab2_piso	tb_piso4
16	lab2_moore	tb_moore_cycle
17	lab2_mealy	tb_mealy_toggle
18	lab2_segment_scan	tb_segment_scan8
실험	XDC	
11	lab2_counter.xdc	
12	lab2_clock_divider.xdc	
13	lab2_register.xdc	
14	lab2_shift_register.xdc	
15	lab2_piso.xdc	
16	lab2_moore.xdc	
17	lab2_mealy.xdc	
18	lab2_segment_scan.xdc	

실험별 Vivado 결과 커밋

실험 11에서 18까지 모두 GitHub에 동시에 커밋을 진행하였고, Vivado 결과 커밋은 그에 따라 모두 동일하다.

커밋 해시: [1c44fc7092a9451e124847c3c8cf73126d6ca7c4](#)

장치 프로그램 확인

각 회로의 구현과 Bitstream 생성을 완료한 뒤 Combo II-DLD S75 FPGA 보드에 프로그램하였다. 이후 스위치와 버튼 입력에 따른 LED 및 7세그먼트 출력을 사진과 영상으로 기록하는 방식으로 실험을 진행하였다.

4. 실험11 결과 · 업 / 다운 카운터

목적 및 설계

4비트 Value를 클럭 상승 에지에서 갱신하는 업 다운 카운터이다. Enable이 1일 때 down=0이면 1 증가하고, down=1이면 1 감소한다. 4비트 연산이므로 F 다음은 다시 0으로 순환하며, rst가 항상 우선시된다.

Vivado 실험사진

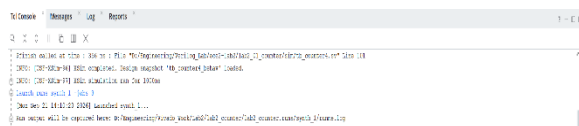


그림 1: 실험 11 Vivado 시뮬레이션 PASS

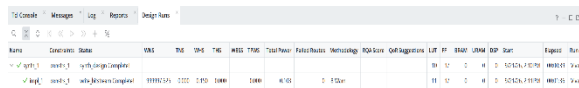


그림 2: 실험 11 카운터 합성 및 구현 완료 결과

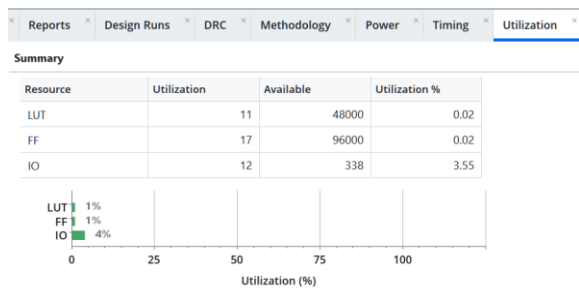


그림 3: 실험 11 카운터 FPGA 자원 사용량



그림 4: 실험 11 카운터 DRC 검사 결과

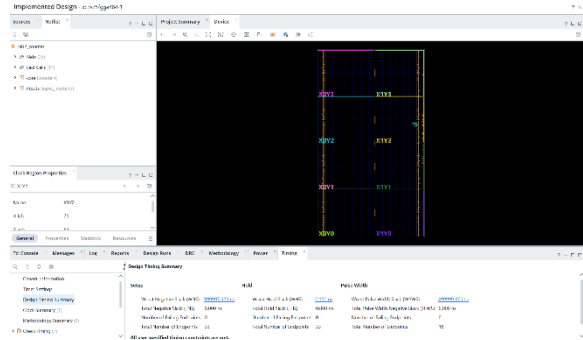


그림 5: 실험 11 카운터 타이밍 분석 결과



그림 6: 실험 11 카운터 Bit Hash 확인

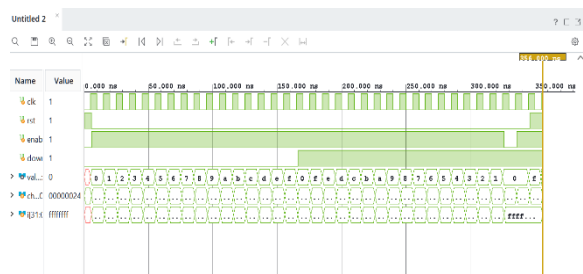


그림 7: 실험 11 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

VS Code와 마찬가지로 36개의 검사가 모두 통과하였다. 파형에서는 리셋 후 Value가 0으로 초기화되고, enable= 1 & down = 0인 구간에서 0부터 F까지 증가한 뒤 다시 0으로 순환하는 모습을 보였다. 그 반대의 구간에서는 F 방향으로 감소하는 모습도 보였다.

FPGA 보드 시연

보드에서는 SW0을 방향 입력으로 사용하고 N8 버튼을 한 번 누를 때 마다 한 단계씩 갱신하였다. SW0=0에서는 LED의 4비트 값이 증가하고 SW0=1에서는 감소했다. 버튼 입력에 따라 LED 값이 증가하고 감소하는 것도 영상에서 확인할 수 있었다.

5. 실험12 결과 · 클럭 분주

목적 및 설계

입력 클럭을 여러 비율로 분주하여 느린 주기의 출력 신호를 생성하는 회로를 설계한다. 카운터가 divisor-1까지 증가하면 다시 0으로 초기화되며, 중간 값에서 divided의 값을 반전시켜 분주된 구형파를 만든다. 또한 카운터가 최종 값에 도달한 한 클럭 동안만 tick을 1로 출력한다.

Vivado 실험사진

```

1  #!/usr/bin/perl
2
3  use strict;
4  use warnings;
5
6  my $file = "data.txt";
7  my $output = "output.txt";
8
9  # Open the input file for reading
10 open(my $fh, "<$file") or die "Could not open file '$file' $!";
11
12 # Read the input file line by line
13 while (my $line = <$fh) {
14     # Split the line into fields
15     my @fields = split(/\t/, $line);
16
17     # Process the fields (e.g., calculate a sum)
18     my $sum = 0;
19     for my $i (0..$#fields) {
20         $sum += $fields[$i];
21     }
22
23     # Print the sum to the output file
24     print $sum . "\n";
25 }
26
27 # Close the input file
28 close($fh);
29
30 # Open the output file for writing
31 open(my $out_fh, ">$output") or die "Could not open file '$output' $!";
32
33 # Write the output to the file
34 while (my $sum = <STDIN) {
35     print $out_fh "$sum\n";
36 }
37
38 # Close the output file
39 close($out_fh);

```

그림 8: 실험 12 Vivado 시뮬레이션 PASS



그림 9: 실험 12 클록 분주기 합성 및 구현 완료 결과

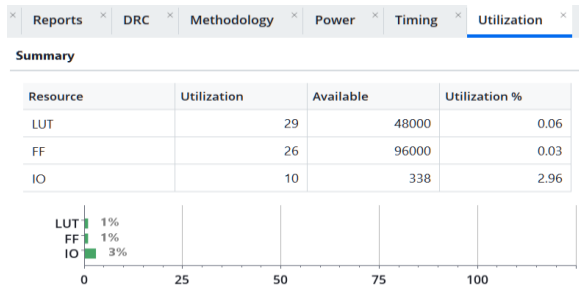


그림 10: 실험 12 클록 분주기 FPGA 자원 사용량



그림 11: 실험 12 클록 분주기 DRC 검사 결과

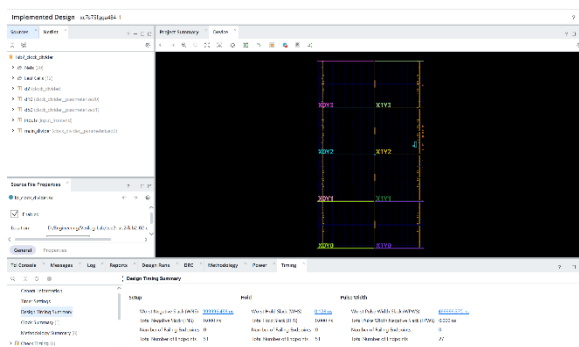


그림 12: 실험 12 클록 분주기 타이밍 분석 결과

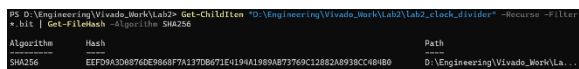


그림 13: 실험 12 클록 분주기 Bit Hash 확인

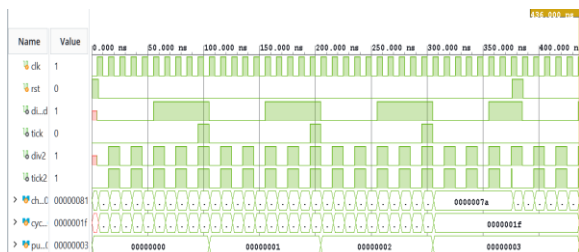


그림 14: 실험 12 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

파형을 살펴보면 카운터가 설정된 분주 값까지 증가한 뒤

반복되었다. Divided는 카운터의 전반부와 후반부에서 각각 0과 1을 유지하여 일정한 듀티비의 구형파를 만들었고, tick은 한 주기의 마지막 클록에서만 짧게 활성화되었다. 따라서 연속적인 분주 출력과 단일 클록 폭의 이벤트 신호가 서로 구분되어 동작함을 확인하였다.

FPGA 보드 시연

1kHz 입력 클록에서 LED0은 2분주된 500Hz 신호이므로 매우 빠르게 점멸하여 계속 켜진 것처럼 보였다. LED3은 1000분주 출력으로 약 1Hz의 주기를 가지므로 켜짐과 꺼짐이 반복되어 육안으로 깜박임을 확인할 수 있었다. 이는 분주비에 따라 출력 주파수가 달라진다는 설계 결과와 일치한다.

6. 실험13 결과 · 레지스터

목적 및 설계

두 개의 4비트 레지스터를 이용하여 병렬 데이터를 저장하고 전달하는 회로를 설계하였다. load 신호가 활성화되면 입력 data_in을 첫 번째 레지스터 stored에 저장하고, transfer 신호가 활성화되면 stored의 값을 두 번째 레지스터 value로 전달하도록 구성하였다. 따라서 입력 데이터의 저장 단계와 출력으로의 전달 단계를 분리할 수 있다.

Vivado 실험사진

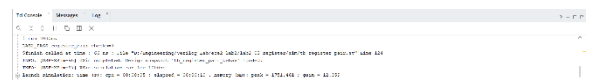


그림 15: 실험 13 Vivado 시뮬레이션 PASS

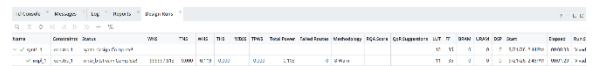


그림 16: 실험 13 레지스터 합성 및 구현 완료 결과

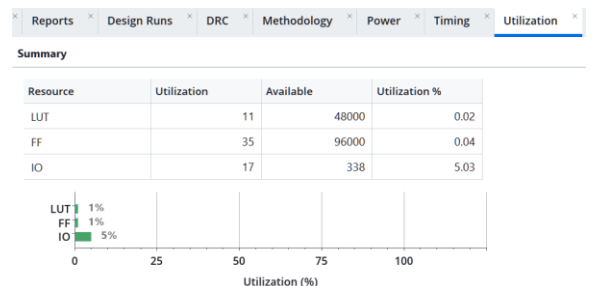


그림 17: 실험 13 레지스터 FPGA 자원 사용량

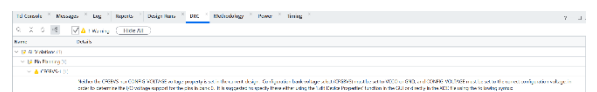


그림 18: 실험 13 레지스터 DRC 검사 결과

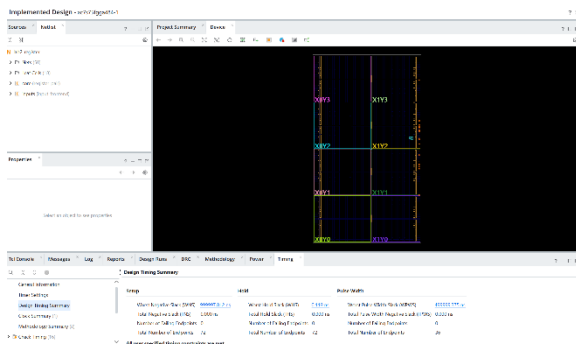


그림 19: 실험 13 레지스터 타이밍 분석 결과



그림 20: 실험 13 레지스터 Bit Hash 확인

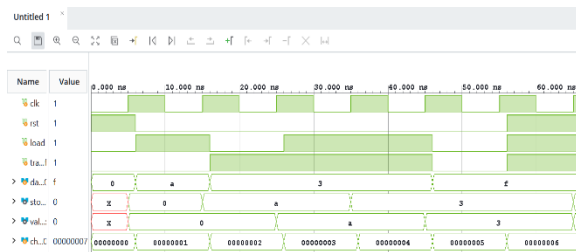


그림 21: 실험 13 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

리셋 이후 load가 활성화된 클럭 상승 에지에서 stored가 data_in 값으로 변경되었다. 이후 transfer가 활성화되면 value가 바로 입력 값을 받는 것이 아니라, 앞서 stored에 저장된 값을 받아 변경되었다. 예를 들어 입력 값이 A, 3, F로 바뀌더라도 load와 transfer가 발생하기 전에는 각 레지스터의 값이 유지되었다. 이를 통해 두 레지스터가 독립적으로 동작함을 확인하였다.

FPGA 보드 시연

스위치 SW[7:4]로 4비트 입력 데이터를 설정하고, 버튼 입력과 SW0을 이용해 Load 동작을 수행하였다. 이때 상위 LED에 저장 레지스터 값이 나타났다. 이후 SW1을 선택하고 버튼을 눌러 transfer를 수행하면 하위 LED에 같은 값이 표시되었다. 스위치 값만 변경했을 때 출력이 바뀌지 않고, load와 transfer를 거친 뒤에만 값이 이동하는 것을 확인하였다.

7. 실험14 결과 · 시프트 레지스터

목적 및 설계

Enable이 1인 상승 에지마다 value를 오른쪽으로 한 비트 이동시키고 serial_in을 최상위 비트에 넣는다. Enable이 0이면 serial_in이 바뀌더라도 기존 value를 유지한다.

Vivado 실험사진



그림 22: 실험 14 Vivado 시뮬레이션 PASS

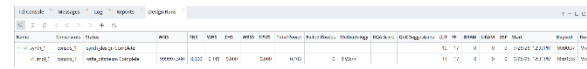


그림 23: 실험 14 shift 레지스터 합성 및 구현 완료 결과

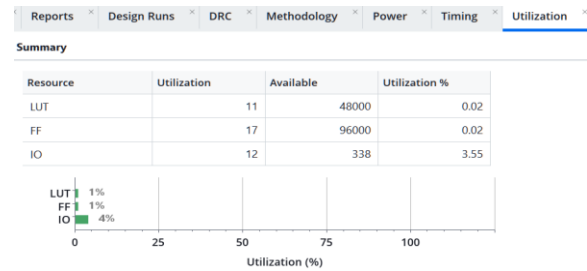


그림 24: 실험 14 shift 레지스터 FPGA 자원 사용량



그림 25: 실험 14 shift 레지스터 DRC 검사 결과

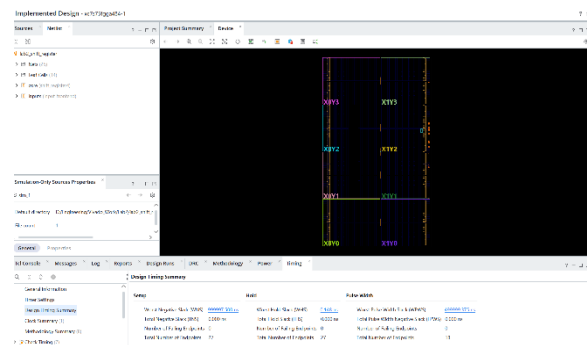


그림 26: 실험 14 shift 레지스터 타이밍 분석 결과

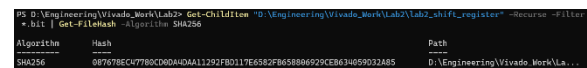


그림 27: 실험 14 shift 레지스터 Bit Hash 확인

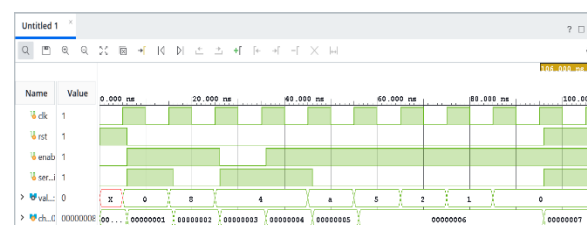


그림 28: 실험 14 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

파형에서 초기값 0에 serial_in=1을 넣으면 8이 되고, 이후 입력에 따라 8, 4, A, 5, 2, 1, 0 순으로 이동하였다. Enable=0 구간에서는 값이 그대로 유지되며 VS Code에서 확인한 예상 결과와 일치하였다.

FPGA 보드 시연

SW7을 직렬 입력으로 두고 버튼을 누를 때 마다 한 비트 씩 이동시켰다. LED[3:0]이 시프트 레지스터 값을 나타내므로 SW7의 값이 LED3로 들어오고 기존 비트들이 LED0 방향으로 이동하는 과정을 확인할 수 있었다.

8. 실험15 결과 · PISO

목적 및 설계

Load가 1이면 4비트 병렬 입력을 value에 저장하고, enable이 1이면 value를 왼쪽으로 이동시키면서 LSB에 0을 채운다. Serial_out은 현재 value의 MSB에 연결되어 병렬 데이터가 MSB부터 순서대로 직렬 출력된다. 이는 병렬 입력, 직렬 출력이라는 PISO의 목적에 맞는 설계이다.

Vivado 실험사진



그림 29: 실험 15 Vivado 시뮬레이션 PASS



그림 30: 실험 15 PISO 합성 및 구현 완료 결과

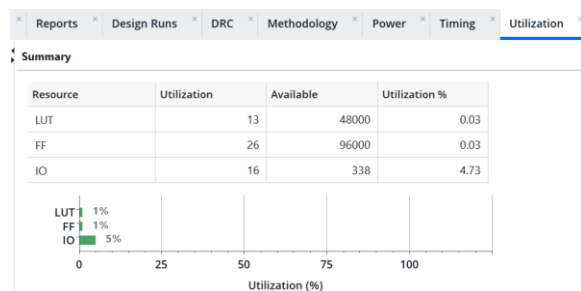


그림 31: 실험 15 PISO FPGA 자원 사용량

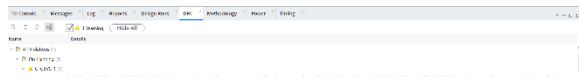


그림 32: 실험 15 PISO DRC 검사 결과

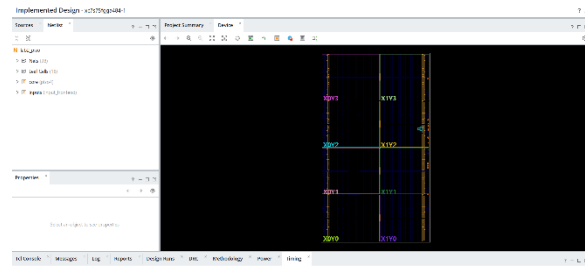


그림 33: 실험 15 PISO 타이밍 분석 결과

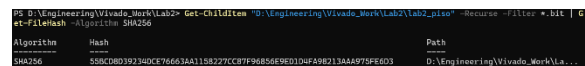


그림 34: 실험 15 PISO Bit Hash 확인

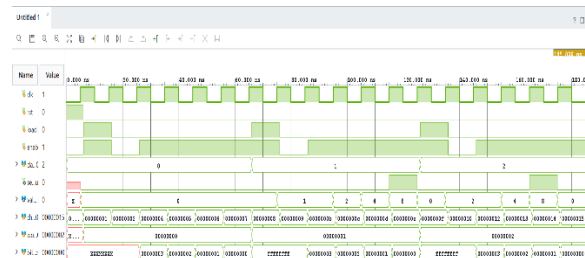


그림 35: 실험 15 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

리셋 후 load=1인 클록 상승 에지에서 data_in의 4비트 값이 value에 병렬 저장되었다. 이후 enable=1인 상승 에지마다 value는 {value[2:0], 1'b0}으로 이동하였다. 예를 들어 입력값 1을 저장한 뒤 value는 1→2→4→8→0으로 변하였으며, value=8인 구간에서 serial_out=value[3]가 1로 나타났다. 따라서 저장된 데이터가 MSB부터 순서대로 직렬 출력되는 것을 확인하였다. 또한 load와 enable이 동시에 활성화된 경우에는 load가 우선하여 병렬 입력이 저장되었다.

FPGA 보드 시연

실험 15의 보드 입력 순서와 LED 출력 변화는 12절의 사진 및 시연 영상에서 자세히 분석하였다.

9. 실험16 결과 · Moore 상태 머신

목적 및 설계

상태와 출력이 같은 value에 저장되는 3상태 Moore 회로이다. Enable과 advance가 모두 1인 상승 에지에서 S0, S1, S2, S0 순서로 전이하고, 그 외에는 현재 상태를 유지하는 회로를 만든다.

Vivado 실험사진



그림 36: 실험 16 Vivado 시뮬레이션 PASS

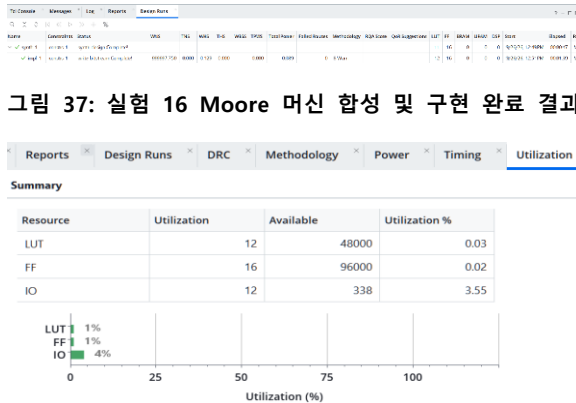


그림 37: 실험 16 Moore 머신 합성 및 구현 완료 결과

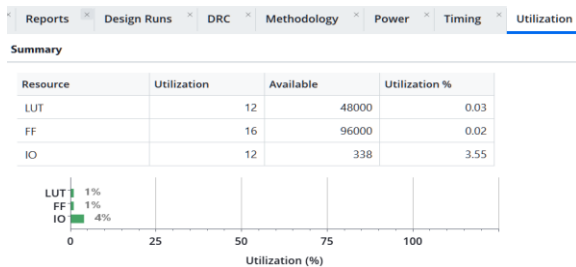


그림 38: 실험 16 Moore 머신 FPGA 자원 사용량

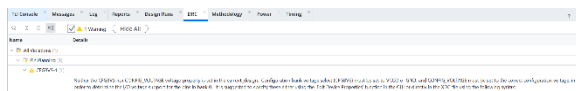


그림 39: 실험 16 Moore 머신 DRC 검사 결과

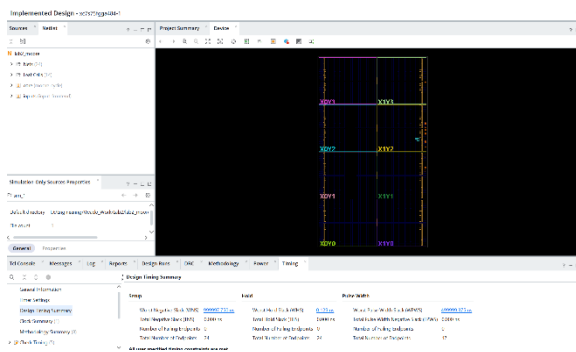


그림 40: 실험 16 Moore 머신 타이밍 분석 결과

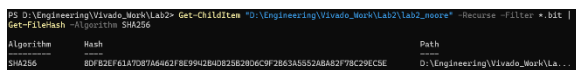


그림 41: 실험 16 Moore 머신 Bit Hash 확인

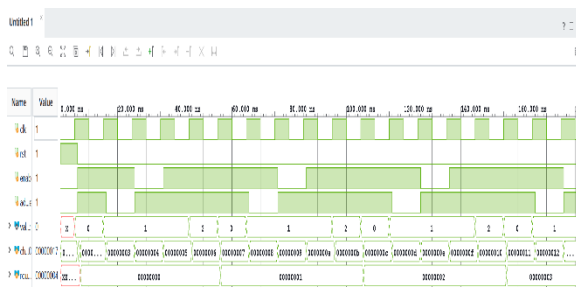


그림 42: 실험 16 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

파형을 살펴보면 value는 유효한 상승 에지마다 0, 1, 2, 0으로 순환하였다. Advance 또는 enable이 0인 구간에서 상태가 바뀌지 않았고, 출력은 현재 상태에만 대응되는 것이 Moore 머신의 동작을 정확하게 보여준다.

FPGA 보드 시연

SW7을 advance로 사용하고 버튼 입력을 enable 펄스로 사용하였다. SW7=1일 때 버튼을 누르면 LED[1:0]이 00, 01, 10, 00 순서로 변했고, SW7=0에서는 버튼을 눌러도 상태가 유지되었다.

10. 실험17 결과 · Mealy 상태 머신

목적 및 설계

Bit_in=1이고 enable이 인가된 상승 에지에서 state가 반전된다. 출력 value는 조합논리로 계산되어 bit_in=0이면 00, 1이면 state=0에서 10, state=1에서 01이 된다. 따라서 입력 변화가 클럭 에지 사이에서도 출력에 반영될 수 있는 Mealy 머신을 보여준다.

Vivado 실험사진



그림 43: 실험 17 Vivado 시뮬레이션 PASS

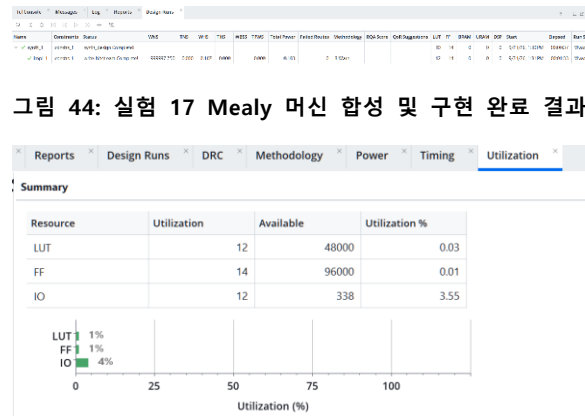


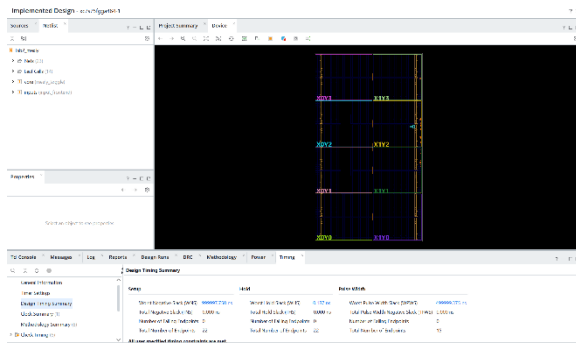
그림 44: 실험 17 Mealy 머신 합성 및 구현 완료 결과



그림 45: 실험 17 Mealy 머신 FPGA 자원 사용량



그림 46: 실험 17 Mealy 머신 DRC 검사 결과



Vivado 시뮬레이션 및 파형 비교

파형에서 index가 0부터 7까지 순차적으로 변화하고, 이에 따라 select가 한 비트씩 이동하였다. 동시에 선택된 자리의 4비트 값에 맞춰 segments 값도 변경되었다. blank 구간에서는 select=8'b00000000이 되어 모든 자리가 잠시 꺼졌으며, 그 다음 자리의 데이터와 선택 신호가 안정된 뒤 다시 표시되었다. 따라서 한 번에 하나의 자리만 선택되는 스캔 동작을 확인하였다.

FPGA 보드 시연

보드의 8자리 7세그먼트에는 7654321과 스위치 입력 값이 함께 표시되었다. 스위치 SW[7:4] 값을 변경하면 COM[7]에 연결된 자리만 변하고, 나머지 일곱 자리는 고정된 숫자를 유지하였다. 회로는 한 자리씩 빠르게 순환 선택하지만, 스캔 속도가 충분히 빨라 모든 자리가 동시에 점등된 것처럼 관찰되었다.

12. FPGA 시연 · 실험 15 - PISO

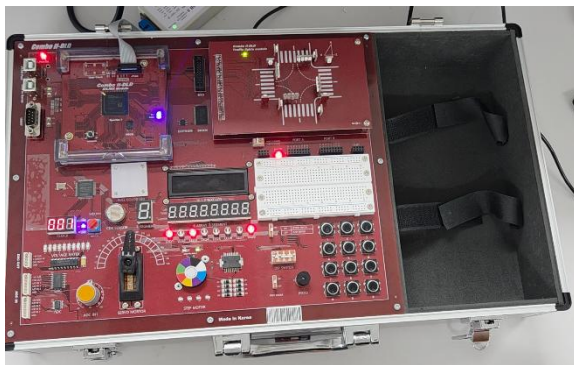


그림 57: 실험 15 PISO FPGA 보드 시연

SW[7:4]를 병렬 입력으로 사용하였다. SW[0]=1에서 버튼을 누르면 입력 값이 value에 저장되고, SW[0]=0에서 버튼을 누르면 저장 값이 한 비트씩 이동하였다. LED[7:4]에는 현재 value가 표시되고 LED[0]에는 serial_out이 표시되었다. 예를 들어 1010을 저장한 뒤 LED0에서 1, 0, 1, 0 순서의 직렬 출력을 확인하였다.

현재 시연 사진에서도 1010을 입력 값으로 하였을 때 직렬 출력이 처음 출력인 1인 상황이다. 이후 SW[0]=0에서 버튼을 누르면 입력 값은 왼쪽으로 이동하여 (0100)4, (1000)8, (0000)0 순으로 변하고 출력도 0, 1, 0, 0 순으로 바뀐다.

시연 영상: [실험 15 PISO 시연 영상](#)

13. 그 외 실험별 시연 사진

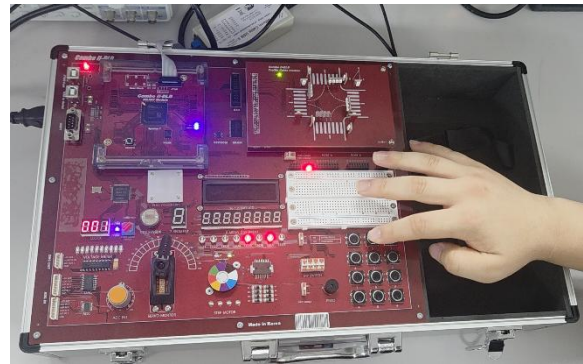


그림 58: 실험 11 FPGA 보드 시연

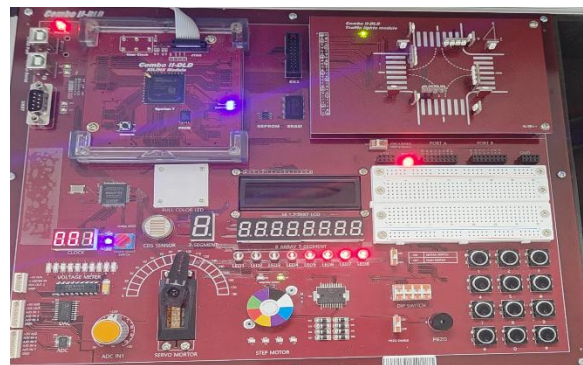


그림 59: 실험 12 FPGA 보드 시연

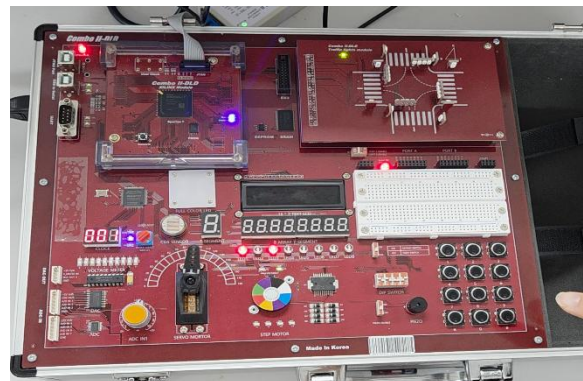


그림 60: 실험 13 FPGA 보드 시연

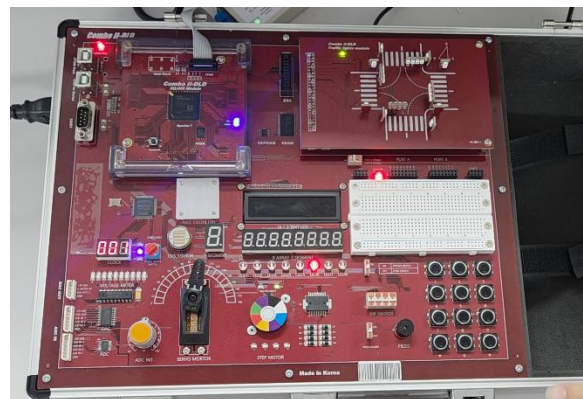


그림 61: 실험 14 FPGA 보드 시연

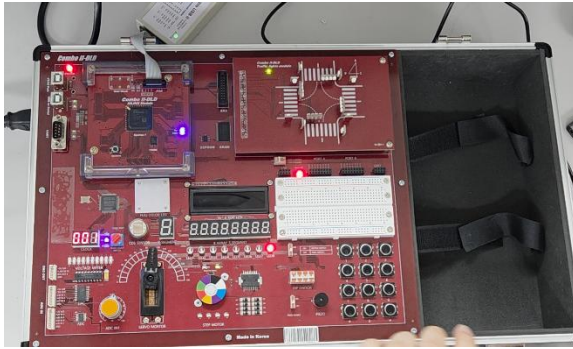


그림 62: 실험 16 FPGA 보드 시연

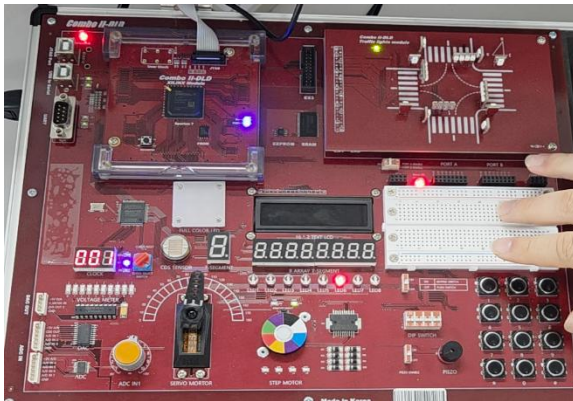


그림 63: 실험 17 FPGA 보드 시연

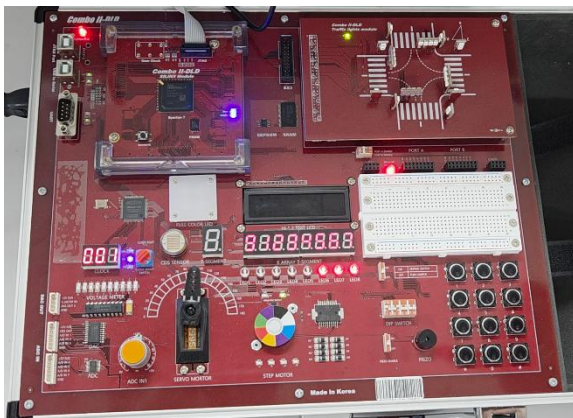


그림 64: 실험 18 FPGA 보드 시연

14. 문제 발생 및 해결 과정

RTL / 테스트 벤치 등록 및 top 설정 확인

보드 구현용 top 모듈과 테스트 벤치 top 모듈을 혼동하면 시뮬레이션이 정상적으로 실행되지 않을 수 있다. 따라서 각 실험에서 Design Top과 Simulation Top을 분리해 등록하고, Vivado에서 테스트 벤치 모듈을 Simulation Top으로 지정하여 해결하였다. 기본적으로 테스트 벤치가 Top에 설정되어 있지 않으므로 반드시 변경해주어야 시뮬레이션 결과가 정확하게 나타난다.

실험 18 시뮬레이션 종료 시각

Vivado 시뮬레이션을 실행하였을 때 기본 실행 시간인 1000 ns에서는 검사가 완료된다. 하지만 실험 18번의 시뮬레이션 총 수행 시간은 1306ns이다. 따라서 run all을 실행해야지만 로그에 finish와 함께 pass 문구가 뜨는 것을 확인할 수 있고, 정확한 시뮬레이션 결과를 얻을 수 있다.

15. 예상 값 · Vivado · 보드 결과 비교

실험	검사 수	Vivado 결과	보드시연	최종
11	36	VS Code동일	확인	정상
12	129	VS Code동일	확인	정상
13	7	VS Code동일	확인	정상
14	8	VS Code동일	확인	정상
15	114	VS Code동일	선정확인	정상
16	23	VS Code동일	확인	정상
17	11	VS Code동일	확인	정상
18	194	VS Code동일	확인	정상

16. Bitstream 해시 표

실험	Bitstream	SHA-256
11	lab2_counter.bit	18D473D...
12	lab2_clock_divider.bit	EEFD9A3...
13	lab2_register.bit	2DDBF46...
14	lab2_shift_register.bit	087678E...
15	lab2_piso.bit	55BCD8D...
16	lab2_moore.bit	8DFB2EF...
17	lab2_mealy.bit	E492086...
18	lab2_segment_scan.bit	F7A5709...

17. 결론

본 실험에서는 업·다운 카운터, 클럭 분주기, 레지스터, 시프트 레지스터, PISO, Moore 및 Mealy 상태기계, 7세그먼트 스캔 회로를 Verilog HDL로 구현하였다. 각 회로는 VS Code와 Vivado XSim에서 동일한 테스트벤치 결과를 보였다. 합성·구현·bitstream 생성 이후 FPGA 보드에서 스위치와 버튼 입력에 따른 LED 및 7세그먼트 출력을 확인하였다. 특히 시뮬레이션 결과뿐만 아니라 LED와 세그먼트와 같이 눈으로 확인할 수 있는 출력을 통해 결과를 확인하였을 때 더 직관적으로 회로의 동작을 볼 수 있었다. 이를 통해 순차논리회로에서 클럭, 리셋, enable, 상태 저장 및 조합 출력이 수행하는 역할을 실제 설계와 보드 동작을 통해 확인할 수 있었다.

참고 문헌

[1] M. Morris Mano, Charles R. Kime, and Tom Martin, *Logic and Computer Design*

Fundamentals, 5th ed., Global Edition,
Pearson, 2016.